Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Dersin ilk yarısında Arş. Grv. Begüm Özbay "Yapısal Programlama Uygulama-2"yi (24.03.2025) yaptı: üç soru ve Lab 1'in çözümü. Ardından Ahmet Hoca ek ders olarak `realloc`, düzensiz (jagged) matris, fonksiyondan dizi döndürme ve `main` argümanlarını anlattı.
- Uygulama Soru 1: Üç zarın toplam olasılık matrisi
  - Üç zar birlikte atılıyor. Matrisin birinci sütununa olası toplamlar (3-18), ikinci sütuna bu toplamın olasılığı yazılacak. Matris dinamik bellekle tanımlanacak.
  - Şartlar: birinci sütundaki toplamlar tekrar etmemeli (benzersiz). Olasılıklar elle (hard-coded) yazılmamalı, döngü ve aritmetikle hesaplanmalı.
  - Fonksiyonlar:
    - `float **initialize_mat(int nrow, int ncolumn)`: satır işaretçileri için `malloc(nrow * sizeof(float*))`, her satır için `calloc(ncolumn, sizeof(float))`. `NULL` kontrolü yapılır, bellek ayrılmadıysa `exit(1)`. Birinci sütuna `i + 3` yazılır (satır 0 → toplam 3).
    - `calc_prob(float **mat)`: iç içe üç `for` (her biri 1'den 6'ya, `<= 6`). `toplam = i + j + k`, `mat[toplam - 3][1] += 1.0 / 216`.
    - `print_probs` ve `free_mat` (önce her satır, sonra satır dizisi `free`).
  - `main`: `nrow = 16` (3-18 arası 16 farklı toplam), `ncolumn = 2`. Önce olasılıklar sıfır olarak yazdırılır (`calloc` sıfırladığı için), sonra hesaplanıp tekrar yazdırılır.
  - Fonksiyonun `float **` döndürmesinin nedeni: iki boyutlu dinamik dizi, satırları gösteren işaretçilerden oluşan bir işaretçi dizisidir.
  - Çıktı: 3 → 0.004630, 4 → 0.013889, 5 → 0.027778, ..., 10 ve 11 → 0.125, ..., 18 → 0.004630.
- Uygulama Soru 2: Collatz dizisi
  - $f(n) = \frac{n}{2}$ (n çift), $f(n) = 3n + 1$ (n tek). Collatz problemi her pozitif tam sayıdan başlanınca dizinin sonunda 4, 2, 1 döngüsüne girip 1'e ulaşacağını iddia eder. Hâlâ kanıtlanmamış bir problemdir.
  - Örnek: 13 → 40 → 20 → ... → 1.
  - İstenenler: kullanıcıdan pozitif n al (`do-while` ile `n <= 0` ise tekrar sor). `calloc` ile 2 elemanlık dizi aç. 1'e ulaşana kadar (`while (n != 1)`) her yeni değeri diziye ekle. Dizi dolunca (`i == current_length`) boyutu iki katına çıkar ve `realloc` yap. Sonunda diziyi yazdır ve `free` et.
  - `int f(int n)` fonksiyonu `n % 2 == 0` kontrolüyle yazıldı.
  - `main`'den sonra yazılan fonksiyonlar için prototip gerekir (öğrenciler hatırlattı). Derste ayrıca `print`/`printf` yazım hatası ve eksik `return 0` düzeltildi.
  - Deneme: 8 → 8, 4, 2, 1. 27 için dizi çok uzar.
- Uygulama Soru 3: Toptancı faturası
  - Ürünler: soda (10 TL), meyve suyu (15 TL), kola (20 TL), su (5 TL). Kullanıcı her üründen kaç koli aldığını girer.
  - `generate_invoice(quantities, product_names, unit_prices)`: her ürün için `amount = quantity * unit_price` hesaplanır ve `total_amount`'a eklenir. Her fatura satırı için `malloc(MAX_STRING_LENGTH)` ile yer açılır, satır `sprintf` ile yazılır. Son satır toplam tutardır.
  - `print_invoice` ve `free_invoice` (her satır, sonra satır dizisi). `#define MAX_PRODUCTS 4`, `#define MAX_STRING_LENGTH 100`.

  - Örnek: 3 soda, 2 meyve suyu, 5 kola, 10 su → 30 + 30 + 100 + 50 = 210 TL.
  - Ahmet Hoca derleme hatası için şunu söyledi: "`int amount` döngü içinde tanımlanmaz, yukarıda tanımlarız. C++'tan gelen bir alışkanlık."
- Lab 1 çözümü
  - İki adet $2 \times n$ matris. İlk satırda benzersiz tam sayılar, ikinci satırda bunlara karşılık gelen değerler var. İki matrisin ilk satırlarında ortak olan sayılar bulunup karşılık değerleri toplanacak.
  - Örnek: 1. matris ilk satır 1, 5 / ikinci satır 7, 2. 2. matris ilk satır 2, 3, 5 / ikinci satır 4, 8, 1. Ortak sayı 5: 2 + 1 = 3.
  - `setlocale` ile Türkçe karakter desteği. Benzersizlik kontrolü bir bayrak (`flag`) değişkeniyle yapılır: girilen sayı daha önce varsa bayrak 1 olur ve "matrisin ilk satırı benzersiz olmalıdır" uyarısı verilir.
  - Ortak sayılar iç içe iki döngüyle bulunur.
- Ek ders: `realloc`
  - Daha önce `malloc` veya `calloc` ile açılmış bir alanın boyutunu program çalışırken değiştirir: `realloc(eski_isaretci, yeni_boyut)`.
  - Durumlar:
    * Mevcut yerin devamında yeterli boş alan varsa alan orada genişletilir.
    * Yoksa heap'te başka bir yerde yeni alan bulunur, eski veriler oraya kopyalanır, eski alan serbest bırakılır. Büyük isteklerdeki bekleme bu kopyalamadan kaynaklanır.
    * Hiç yer bulunamazsa `NULL` döner. Bu durumda eski alan olduğu gibi (intact) kalır.
  - Hoca, sürekli büyüyen bir diziyi `do-while` ile denedi (`Y` girildikçe yeni boyut istenir). Çok büyük bir istekte `NULL` döndü.
  - **Güvenli kullanım:** sonucu önce geçici bir işaretçiye alın.
    * `tmp = realloc(array, yeni_boyut); if (tmp == NULL) { hata; } else { array = tmp; }`
    * Doğrudan `array = realloc(array, ...)` yazarsanız ve `NULL` dönerse, eski alanın adresi kaybolur. Önceki bütün veriler erişilemez hâle gelir (bellek sızıntısı).
  - Negatif değer girilerek dizi küçültülebilir. Yeni boyut 0 olunca alan serbest bırakıldı ve `NULL` döndü.
  - Hoca debugger'da `tmp` ve `array`'in aynı adresi gösterdiğini gösterdi.
- Ek ders: düzensiz (jagged / ragged) matris
  - Her satırın sütun sayısı farklıdır. C'de hazır böyle bir yapı yoktur, işaretçilerle kurulur.
  - Satır sayısı n alınır. Yardımcı bir dizi her satırın sütun sayısını tutar (örnekte `rand() % n + 1`).
  - `matrix = malloc(n * sizeof(int*))`, sonra her satır için `matrix[i] = malloc(array[i] * sizeof(int))`.
  - Doldururken iç döngünün sınırı `array[i]`'dir (o satırın sütun sayısı).
  - Serbest bırakma: önce her `matrix[i]`, sonra `matrix`.
  - `print_list(int *list, int size)` fonksiyonuna satır `matrix[i]` (veya `*(matrix + i)`) ve boyut `array[i]` olarak gönderilir. `matrix[i]` ile `*(matrix + i)` aynı şeydir.
  - Ödev: hem düzensiz hem `realloc` ile satır boyutları değiştirilebilen iki boyutlu bir matris kodu yazın (satır boyutlarını tekrar sorun, yazdırma fonksiyonu yapın).
- Fonksiyonlar ve dizi döndürme
  - `main`'den önce yazılan fonksiyonlar için prototip gerekmez. Sonra yazılırsa prototip (bildirim) yazılır.
  - Dizi parametresi `int a[]` veya `int *a` olarak yazılabilir. `int *a` daha geneldir: her dizi adı bir adres gibi kullanılır, ama her işaretçi bir dizi değildir.
  - C'de fonksiyon doğrudan dizi döndüremez. Bunun yerine işaretçi döndürülür.
  - **Yanlış:** fonksiyon içinde `int d[5]` tanımlayıp `return d;`. Sabit uzunluklu diziler stack'te oluşturulur. Fonksiyon bitince bu alan artık geçerli değildir. Derleyici uyarı verir. "Kod çalıştı" demek doğru olduğu anlamına gelmez, bazen çalışmaz. Hoca bu çözümün sıfır puan bile almayacağını söyledi.
  - **Doğru:** `int *d = malloc(5 * sizeof(int)); return d;`. Heap'teki alan fonksiyon bitince de yaşar.
  - Örnek: iki diziyi birleştiren (`merge`/`concatenate`) fonksiyon `malloc(size1 + size2)` ile yer açıp önce birinci, sonra ikinci diziyi kopyalar ve işaretçiyi döndürür.
  - Slayttaki ikinci versiyonda sonuç dizisi çağıran yerde sabit uzunluklu olarak oluştu-

ruluyor. Hoca bunun doğru olmadığını söyledi: diziler `malloc` ile oluşturulup gönderilirse yaklaşım doğrudur.
  - Ev ödevi: iki sıralı diziyi dinamik olarak birleştiren program.
- `main` argümanları
  - `int main(int argc, char *argv[])`. `argc` argüman sayısı, `argv` argümanları tutan `char*` dizisi.
  - `argv[0]` her zaman programın adıdır (tam yoluyla). Programa 4 argüman verilirse `argc` 5 olur.
  - Dev-C++'ta "Execute → Parameters" ile argüman verilebilir. Komut satırından da verilir: `MainArguments.exe deneme abc 123`.
  - `cd`, `dir /w`, `dir /p` gibi komutlar da dışarıdan argüman alan programlardır.
  - Argümanlar her zaman string gelir. Sayı olarak kullanmak için `atoi` (int), `atof` (double) ile çevrilir.
  - Örnek program: argüman yoksa kullanım bilgisi yazdırılır ("En az 3 argüman girilmelidir. Arg-1 öğrenci sayısı, Arg-2 ders sayısı, Arg-3 sınav yeri"). Varsa `yuk = atoi(argv[1]) * atoi(argv[2])` hesaplanır. `argv[1] * argv[2]` yazmak derleme hatası verir: iki `char*` çarpılamaz.
  - Sayıya çevrilemeyen argüman (ör. "B5", "abc") `atoi` ile 0 olur ve sonuç yanlış çıkar. Programın kullanım kurallarını kullanıcıya bildirmek gerekir.
  - `getopt` gibi fonksiyonlar komut satırı seçeneklerini (`-a`, `-b` gibi) daha düzenli işlemeyi sağlar. Sonraki derste değinilecek.
- Sonraki ders (çarşamba): struct'lar.

Hocanın Özellikle Vurguladığı Kısımlar

- `realloc` sonucunu her zaman geçici bir işaretçide kontrol edin, sonra asıl işaretçiye atayın.
- Fonksiyondan yerel (stack'teki) dizinin adresini döndürmeyin. Dinamik bellek kullanın. Bu hata sınavda puan almaz.
- `malloc` ile oluşturulan her şey heap'tedir. Sabit uzunluklu diziler stack'tedir. İşaretçi döndürmek alanı heap'e taşımaz.
- Düzensiz matriste iç döngünün sınırı o satırın kendi boyutudur.
- `main` argümanları string'dir, sayıya çevirmek gerekir.
- Değişken tanımlarını döngü içine değil, bloğun başına yazın (eski C alışkanlığı).

Kısa Tekrar Notları

- 2B dinamik matris: `T **m = malloc(r * sizeof(T*)); for (i...) m[i] = calloc(c, sizeof(T));` Serbest bırakma ters sırada.
- Güvenli büyütme: `tmp = realloc(p, n * sizeof *p); if (tmp) p = tmp;`
- Kapasite iki katına çıkarma: `if (i == cap) { cap *= 2; realloc; }`
- `matrix[i] == *(matrix + i)`.
- `argc`, `argv[0]` = program adı, `atoi`/`atof` ile dönüşüm.
- Üç zar: toplamların sayıları 1, 3, 6, 10, 15, 21, 25, 27, 27, 25, 21, 15, 10, 6, 3, 1 (toplam 216).

Detaylı Açıklamalar

Ders sonrası notlar

- **Kodların kontrolü:** Zar olasılık programı ve Collatz programı ders sonrasında gcc ile derlenip çalıştırıldı. Zar olasılıkları slayttaki tabloyla aynıdır ve toplamları 1'dir. Collatz dizisi 27 için 112 eleman uzunluğundadır, en büyük değer 9232'dir. Fatura örneğinin toplamı 210 TL'dir.
- **`realloc` hata durumunda ne döndürür?** Derste "negatif bir değer döndürecek" dendi. `realloc` başarısız olunca `NULL` döndürür, negatif sayı döndürmez.
- **Collatz kodunda `realloc`:** Uygulamada `A = (int*) realloc(A, ...)` doğrudan aynı işaretçiye atandı ve `NULL` kontrolü yapılmadı. Hocanın ek derste anlattığı geçici işaretçi

(`tmp`) yöntemi daha güvenlidir. Ayrıca dizi elemanları ekrana aralarında boşluk olmadan (`"%d"`) yazdırılıyordu; `"%d "` kullanılmalıdır.

- **`realloc(p, 0)`:** Derste yeni boyut 0 olunca alanın serbest bırakılıp `NULL` döndüğü görüldü. Bu davranış derleyiciye/kütüphaneye bağlıdır (C23'te tanımsız davranış sayılır). Bir alanı serbest bırakmak için `free` kullanın, `realloc(p, 0)`'a güvenmeyin.
- **İşaretçi boyutu:** Derste "8 byte 32 bit demek, 2 üzeri 32 = 4 GB adresleyebilir" dendi. 8 byte = 64 bittir. 64 bitlik sistemlerde işaretçiler 8 byte'tır (gcc ile `sizeof(int*) = 8` doğrulandı) ve teorik olarak 2^{64} adres gösterebilir. 4 GB sınırı 4 byte'lık (32 bit) işaretçilere aittir.
- **Döngü içinde değişken tanımı:** Hocanın "C'de döngü içinde tanımlanmaz" uyarısı eski C standardı (C89/C90) için geçerlidir; orada `for (int i = 0; ...)` derleme hatası verir. C99 ve sonrasında döngü gövdesinde ve `for` başlığında değişken tanımlamak geçerlidir. Dev-C++'ın eski sürümleri varsayılan olarak C90 kipinde derleyebildiği için hata görülebilir. Sınavda hocanın tercihine uymak güvenlidir.
- **Yerel dizi döndürme:** Ders sonrası gcc ile denendi: derleyici "function returns address of local variable" uyarısı verir ve güncel gcc bu durumda fonksiyondan `NULL` döndürecek şekilde kod üretir. Yani bu hata bazı derleyicilerde "çalışıyor gibi", bazılarında hemen çöken bir program verir.
- **"Her dizi bir işaretçidir":** Dizi adı çoğu ifadede ilk elemanın adresine dönüşür, bu yüzden fonksiyona `int *` olarak gönderilebilir. Ama dizi ile işaretçi aynı şey değildir: `sizeof(dizi)` bütün dizinin boyutunu, `sizeof(isaretci)` 8'i verir ve dizi adına yeni adres atanamaz.
- **`main` argüman kontrolü:** Derste yalnızca `argc == 1` kontrol edildi. Program `argv[3]`'e eriştiği için en az 3 argüman gerekir: kontrol `if (argc < 4)` olmalıdır. Aksi hâlde 1-2 argümanla çalıştırılınca var olmayan bir argümana erişilir. Ayrıca `atoi("5B")` 5, `atoi("B5")` ve `atoi("abc")` 0 döndürür (gcc ile denendi). Hatalı girişi ayırt etmek için `strtol` kullanılabilir.
- **`fflush(stdin)`:** Hem uygulamada hem ek derste `scanf` sonrası kalan Enter karakterini temizlemek için `fflush(stdin)` kullanıldı. Standart C'de `fflush` yalnızca çıkış akışları için tanımlıdır; `stdin` için davranış Windows'a özgüdür. Taşınabilir yol `scanf(" %c", &c)` (`%c`'den önce boşluk) veya `while (getchar() != '\n');` kullanmaktır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.